UNIVERSITI TUNKU ABDUL RAHMAN
LEE KONG CHIAN FACULTY OF ENGINEERING AND SCIENCE
LABORATORY SAFETY RULES

The following rules must be obeyed by all students in the Computer laboratories of the faculty. Willful or repeated inadvertent noncompliance may result in dismissal or suspension from the laboratories.

I. No entry without permission:

i) Outsiders are not allowed to enter the laboratory without permission.

ii) No student is allowed to enter the laboratory unless permission has been given by a laboratory staff or a lecturer.

II. At work in the laboratory:

i) The computer facilities at the laboratory are to be used for learning purposes only.

ii) Wear your I.D. cards at all times in a proper visible position whenever you come into the computer laboratory.

iii) It is compulsory to follow dress code of UTAR
   a) Covered shoes (no slippers or flip flops)
   b) Long pants and tidy attire

iv) Do not eat or drink in the laboratory.

v) Do not tamper with any computers, printers, or any other associated equipment. If there are problems or faults, report immediately to the staff at the laboratory.

vi) Do not remove any equipment or document from the computer laboratories.

vii) Do not alter the default hardware and software setup of the computers and printers in the computer laboratories.

viii) Do not copy, rename, change, examine or delete files or information belonging to any other user of the University.

ix) Do not install any unauthorized or unlicensed software into the PC of computer laboratories.

x) Do not make any noise, chit-chat, hold group discussions or disturb other users at the laboratory in any way.

xi) Silent your hand phone when you are in the computer laboratories.

xii) Any equipment broken or damaged must be reported immediately to the laboratory staff.

III. Before leaving the laboratory:

i) No student is allowed to take away any equipment, tools or other property of the laboratory without permission.

ii) Please check and remember to take all your personal belongings such as handphone, wallet, pendrive, hard disk, book, bottle, sweeter, and etc.

iii) Lab staffs are not responsible for lost of personal belongings.

Universiti Tunku Abdul Rahman

Faculty	**Lee Kong Chian Faculty of Engineering and Science**
Department:	**Department of Electrical and Electronics Engineering**
Unit Code and Name	**UEEN 1043 Object-Oriented Concepts and Programming Techniques**
Experiment No.:	**1 / 7**
Title of Experiment:	**IDE and Basic Data Types**
Laboratory Room No. and Name:	**Computer lab**
Experiment Duration (hour):	**3 hours**
Number of Student per Group	**1 student**

Objectives

- To use the Python command console and IDE
- To understand basic data types in Python

Equipment and Materials

Item Description	***Item category**	Quantity estimation (e.g. per set/group of student)
Personal computer	E	1 per student
Python interpreter (Anaconda Spyder)	S	1 per student

***Item category**	
SP	Sample or specimen
C	Consumable
CH	Chemical
W	Labware, glassware, tool, and components
E	Equipment
S	Software

Caution Statement (Optional)

FACULTY OF ENGINEERING AND SCIENCE
UNIVERSITI TUNKU ABDUL RAHMAN

UEEN 1043 Object-Oriented Concepts and Programming Techniques

Practical 1: IDE and Basic Data Types

Objectives

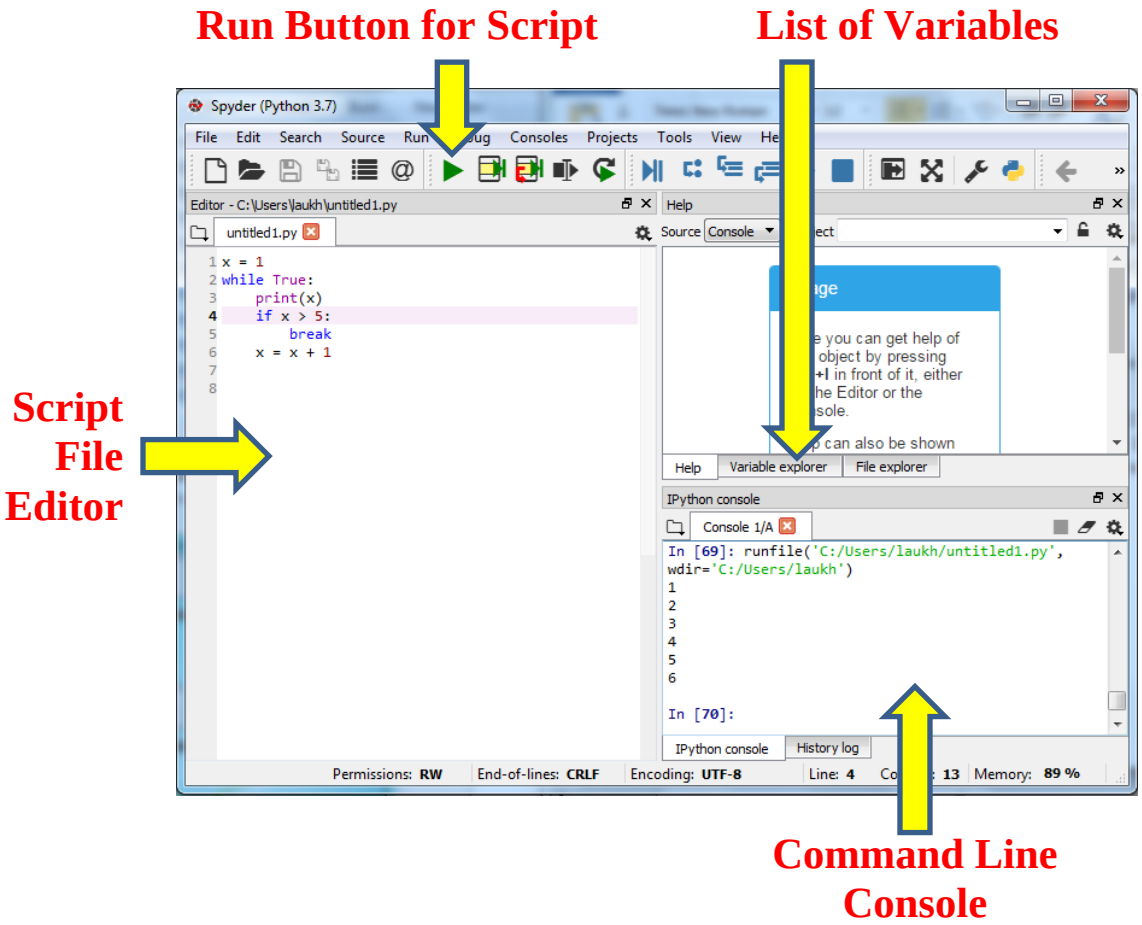
- To use the Python command console and IDE
- To understand basic data types in Python

Equipment

- Computer with a Python interpreter (e.g. Anaconda Spyder)

Introduction

Launch Anaconda Spyder IDE for Python and write codes for the following exercises.



Exercises

1-1 Comments

Comments are texts written on a piece of program which are meaningless to computers, but they help the human readers to understand the program better. Comments are lines starting with the # symbol and extend to the right.

```
#This whole line is a comment, I can type anything I like here!
x = 1   #anything to the right of hash is a comment
```

1-2 Data Types

Data types are values that can be stored as memories and manipulated by Python in its computation. There are many data types in Python, the basic types are: Boolean (True/False), integer (1,2,3,…etc.), float (2.5 etc.), string ('Hello' etc.). Try entering the following lines on the **Command Line Console** and see their line responses:

```
type(True)
type(5)
type(2.5)
type('Hello')
```

1-2-1 Boolean

Type the following, one line at a time, on **console** and understand each output responses:

```
True
False
1==1
1!=1      #or 1<>1
1>2
2>1
1>1
1>=1
True & False
True | False
True | True & False
False & False | True
```

Booleans are useful in if-statements:

```
if 2>1:
    print("True")
else:
    print("Untrue")
```

1-2-2 Numbers

Type the following, one line at a time, on the command line console and understand the outputs:

```
1+5
8-2
9*2
9**2
9**0.5
3/2
3.0/2
3/2.0
7%2
```

There are specific orders in computer arithmetic operations.

```
2+3*5-7/2
```

Type the following, one line at a time, and see how the value of variable x changes.

```
x = 3
x
x = x + 1
x
```

```
Let’s manipulate other types of number:
```

```
r = 4.
r == x
c = 1.-3j
c
c+c
c*c
c*c.conjugate()
type(x)
type(r)
type(c)
```

1-2-3 String

A string is a sentence i.e. array of charactors.

```
t = 'I like PC'
print(t)       #or simply t
t[0]
t[2]
```

```
t[-1]
t[-2]

text = ''' many
lines of
texts '''
print(text)
```

The characters in a string can be indexed.

```
s = '12345678'
s[:3]
s[3:]
s[2:-2]
s[::-1]
s[::2]
```

When strings contain all numbers, they can be converted to a suitable number type.

```
x = s
x = x + 1      #notice the error
x = int(s)
x = x + 1
```

Converting from a number back to string is just as easy.

```
w = str(x+1)
print(w)
```

We can concatenate strings, which means joining them up.

```
s + t
s + t + w
```

There are many useful functions with strings:

```
' u t a r '.find('a')    #return substring index, -1 if none
' u t a r '.strip()      #remove extra spaces in front/back
' u t a r '.replace(' ', '')  #replacing texts
```

Finally, you may clear up variable memories by executing on the console,

```
%reset
```

1-3 List

We can form an array of elements consists of Python data types. Lists are popular in Python.

```
x = [1, 2, 3]
x[0]                        #index into the array
x[1:2]                      #return a shorter sub-array
x[::-1]                     #reversing an array
y = ['a', 'b', 'c']
x + y                       #concatenate/join 2 arrays
```

Converting string to list:

```
s = ' a, b, c, d '
news = s.replace(' ', '')#remove spaces
news.split(',')             #chop up strings by comma into a list
```

Lists are usually accessed element by element through for loops.

```
for e in x:
    print(e)

for e in y:
    print(e)
```

1-4 Keyboard Input

Keyboard input is treated as a string which can be cast to number.

```
s = input('ans: ')
ans: 'Hello'
t = input('no: ')
no: 7
s
t
```

1-5 String format

Type the following codes and understand the code and outputs:

```
a = 'first'
b = 'second'
c = 'third'
d = 'last'
s = a + ' ' + b
s = s + ' %s' %c                #c is the actual argument
s = s + ' {x}'.format(x=d)      #d is the actual argument
```

1-6 File Input and Output

Files are commonly used long term memory in a computer. Files can be created and read in the following manner:

```
f = open('my.txt', 'w')
f.write('a b c d\n')
f.write('1 2 3 4\n')
f.close()

f = open('my.txt', 'r')
lines = f.readlines()    #returns a list of text lines
f.close()
for x in lines:          #step through each element in lines
    print(x)

f = open('my.txt', 'r')
verylong = f.read()      #returns a string of the whole text
f.close()
```

1-7 Debug

Create a Python script file (*.py) with the following lines:

```
print 'This is the start'
x = 0; y = 0
x = 3.5
y = 2*x-4j
x += 1
print 'x is', x
print 'y is %d, y is %d ' %  y
print 'This is the end'
```

Save the file above which contain several code errors. Click on the toolbar ‘Run file (F5)’. Identify the first error which is highlighted by a red cross next to the line number on the script. Read the console’s outputs to understand the source of error and correct for it. Repeat F5 and solve for the next error, and so on until all syntax errors are removed. When there is no more error, step through the program by clicking on the toolbar ‘Debug file (Ctrl+F5)’. Spyder will step from the beginning of the script. The arrow on the console screen points to the next instruction to be executed. You can step through each instruction by clicking on the toolbar ‘Run current line (Ctrl+F10)’. As you step through each line, see the outputs on console, and watch the memory contents in the variable explorer pane. Try to match each change with the program’s semantic. To fast-forward debugging by starting from a specific point in code, you can double click on the line number next to that instruction, and then press Ctrl+F5. Then, step through the codes as usual. To stop debugging, click the stop toolbar ‘Exit Debug (Crtl+Shift+F12)’.

Exercise

It is a must for you to demonstrate how to look for relevant knowledge/solution on the Internet on your own when you want to find answers beyond the lecture notes in order to complete the following.

A1

The input file A1_input.txt has the following content for example:

```
[a,b] , [12, 'y']4['1', 2, '2 3', x]:
[ b, 123, ! ]
```

Write a python program that can read and organize A1_input.txt into the output as shown below. Extra test line inputs may also be added by the examiner to fully test out your code's generality. Design your solution to be simple but general. Hints: Focus on using lists, for-loops, functions readlines, split, and replace. ***DO NOT import*** any library.

```
Extract bracketed contents:
a,b
12, 'y'
'1', 2, '2 3', x
 b, 123, !

Processed contents:
a,b
OneTwo,y
One,Two,TwoThree,x
b,OneTwoThree,!
```